

LLM-Assisted Repository-Level Generation with Structured Spec-Driven Engineering

Shuzhao Feng

McGill University
Montreal, Quebec, Canada
shuzhao.feng@mail.mcgill.ca

Boqi Chen

University of Ottawa
Ottawa, Ontario, Canada
boqi.chen@uottawa.ca

Brett H Meyer

McGill University
Montreal, Quebec, Canada
brett.meyer@mcgill.ca

Gunter Mussbacher

McGill University
Montreal, Quebec, Canada
gunter.mussbacher@mcgill.ca

Abstract

State-of-the-art Large Language Models (LLMs) excel in code generation at the function level. However, the output quality significantly declines when scaling to repository-level systems. Current workflows relying only on natural language prompts suffer from inherent ambiguity and a lack of verifiability. To address this, we propose *structured spec-driven engineering* (SSDE), a paradigm that leverages structured artifacts to guide LLM generation. We argue that structured specifications as LLM inputs make high-quality, repository-level code generation a tangible goal, while at the same time offering superior verifiability, leading to significant potential for improvement. We first investigate the feasibility of this vision through a pilot study generating Model-View-Controller (MVC) business logic for three software systems using five LLMs, and then highlight the potential, challenges, and future roadmap for SSDE.

CCS Concepts

• **Software and its engineering** → **Automatic programming**: *Unified Modeling Language (UML)*; *Abstraction, modeling and modularity*; • **Computing methodologies** → **Machine learning**: *Planning with abstraction and generalization*.

Keywords

Large Language Models (LLMs), Code Generation, Spec-Driven Development, Artificial Intelligence, Model-Driven Engineering, Software Requirements, Domain Models, Gherkin Specifications

ACM Reference Format:

Shuzhao Feng, Boqi Chen, Brett H Meyer, and Gunter Mussbacher. 2026. LLM-Assisted Repository-Level Generation with Structured Spec-Driven Engineering. In *34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion '26)*, July 05–09, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3803437.3805567>

1 Introduction

Recent advances in Large Language Model (LLM)-based code generation and agentic coding assistants motivate new approaches such as *spec-driven engineering* [13], where evolving specifications guide automated implementation by LLM agents. These approaches offer significant potential for fundamental changes in software engineering workflows, which traditionally rely on engineers to convert

requirements and design to implementation either manually or through bespoke transformations.

However, while studies show LLMs perform well at generating isolated functions or files [10, 23], their reliability declines for larger or repository-level systems [32, 34]. In addition, natural language prompts, commonly used in current SDE workflows, are often found to be lossy in communication due to the inherent ambiguity of natural language [17]. Consequently, this makes it more difficult to generate verification material (e.g., software tests) from the natural language prompts to verify whether LLMs' output adheres to the instructions. This limitation is fundamental, and simply augmenting the quantity or verbosity of natural language instructions is unlikely to solve the issue, as it does not address the underlying lack of precision required for reliable communication and verification. We postulate that more structured and maintainable specifications with reduced ambiguity are needed to enable reliable SDE methods.

For reliable and precise communication, traditional software engineering uses structured specification methods such as Gherkin specifications and software models. Gherkin is a structured, non-expert-readable language to specify system behavior through verifiable examples [12], from which engineers can implement software test suites. Similarly, software models provide automatable and compact representations of software systems, where various types of software models (e.g., domain models, state machines) can capture different software concerns at different levels of abstraction.

Previous work at the intersection of LLMs and structured specifications remains narrow, with most work focused on using LLMs to generate Gherkin specifications or software models [6, 9, 36, 39]. Empirical results show that, although using LLMs to automatically produce structured specifications remains largely impractical, LLMs show impressive out-of-the-box ability in inferring software system properties through these specifications [9].

We argue that structured specifications could be the key to consistent and verifiable LLM code generation at system scale, which ultimately aims to step from file-level LLM code generation tasks towards repository-level generation. This paradigm, which we call *structured spec-driven engineering* (SSDE), leverages the structure and verifiability inherent to structured specifications and LLMs' ability to process them for software engineering tasks.

We hypothesize that structured specifications as LLM input can serve as effective intermediaries between high-level requirements and design, and high-quality repository generation, where specifications (i) effectively guide LLM generation and (ii) offer means to verify the generation outcomes. In this context, LLMs reduce the burden on engineers to manually translate specifications into code, fostering effective human-computer collaboration.



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2636-1/2026/07
<https://doi.org/10.1145/3803437.3805567>

To investigate the feasibility of our vision, we generate model-view-controller (MVC) [38] systems in a pilot study, following the SSDE workflow using Gherkin specifications and domain models as inputs. Our study shows that, with SSDE, high-quality automated repository generation is a tangible goal. The initial results also provide insight into the concrete steps required to realize our vision, offering a roadmap for future research, including steps to improve generation quality and better evaluate the viability of SSDE.

In the remainder of this vision paper, Section 2 explains our pilot study, Section 3 presents its results, Section 4 details our insight gained and our roadmap, followed by the conclusion in Section 5 and the acknowledgement in Section 6.

2 Experimental Setup

Our pilot study seeks to demonstrate the feasibility of the *structured spec-driven engineering* (SSDE) workflow and identify challenges for the research roadmap. As detailed in Figure 1, we use Large Language Models (LLMs) to generate Python MVC business logic (i.e., controllers) for software systems. The generated controller interacts with the model layer to form the full backend system, which is then evaluated for its quality. We use different combinations of inputs for business logic generation and compare the quality of the outcome to understand the contribution of each type of input.

Sample Systems. We select existing software systems from GitHub. Our inclusion criteria require that each system: (i) supports the Model-View-Controller (MVC) architecture; (ii) contains an up-to-date domain model or equivalent (e.g., PlantUML [30] diagrams); and (iii) contains an up-to-date Gherkin specification or clearly defined use cases from test suites or design documents. We ultimately selected three systems [18, 28, 35], each from a distinct problem domain. As an overview of the systems' complexity, Table 1 shows some statistics of these systems' domain models and specifications. **Sample LLMs.** We use five different LLMs for our experiment: Claude Sonnet 4.5 by Anthropic [3], Qwen 3 Coder 480B/A35B Instruct by Qwen [33], GPT 5.1 [27] and GPT 5 Nano [26] by OpenAI, and Llama 3.2 3B Instruct by Meta [24]. The choice of LLMs is primarily based on available state-of-the-art (SOTA) models at the time of the experiment, with a spread of open source (Qwen and Llama), closed source (Claude and GPT), large (Claude, Qwen, and GPT 5.1), and small (GPT 5 Nano and Llama) models.

Figure 1 shows an overview of our Structured Spec-Driven Engineering (SSDE) approach applied to the pilot study.

Inputs. Besides the controller template, which we always provide as LLM contextual input to guide generation outcome, we compare four types of specification as contextual input as follows.

Natural Language Specification — the natural language description of the system's purpose, use cases, and constraints. Though written in free form, this specification is *self-contained*, meaning it includes all essential requirements and constraints for the business logic, such as system behavior and value bounds.

Table 1: MVC System Overview

	Symboleo	CheECSEManager	MeetingGroups
Num. Classes	12	17	29
Num. Enum Classes	6	1	12
Num. Attributes	12	48	88
Num. Relationships	26	20	55
Num. Test Cases	134	120	119

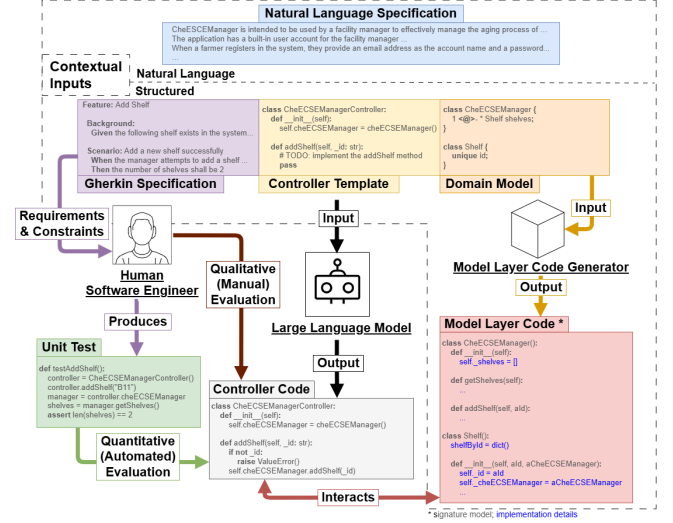


Figure 1: Overview of SSDE Applied to Pilot Study

Gherkin Specification — standardized scenarios following the systematic template of Gherkin. In contrast to the natural language specifications, the Gherkin specifications contain concrete examples describing expected system behavior under varying conditions, which can be directly mapped to executable test cases.

Domain Model — the software model that captures domain concepts and their relationships. Frameworks such as Umple [22] and the Eclipse Modeling Framework (EMF) [37] with its Ecore meta-model promote higher levels of abstraction by providing tooling to automatically generate the model layer code within the MVC framework. We consider two types of domain models for the experiment: Umple and Ecore (through *Emfatic* [15]) models. While Umple and Ecore support embedding business logic, we refrain from doing so to evaluate the LLM's capacity to generate the logic instead.

Signature Model — the model layer code's class and function signatures (API). As shown by the gray text in the red box in Figure 1, we provide the model layer code's class and function signatures as the model to the LLM. The signatures show the model layer API and calling parameters (which are implicit in domain models as they are generated from the domain model) to help LLM identify functions relevant to the business logic. Umple natively provides support for generating Python code, and Ecore does it through *pyecoregen* [29]. **Output.** We prompt the LLM with a combination of selected inputs and ask the LLM to generate the Python business logic that completes the controller template, as shown by the gray box in Figure 1. The generated controller code interacts with the model layer code to form the full backend system. For our pilot study, we use 0-shot prompting [21] with no feedback as the naïve approach.

To reduce LLM's stochastic bias, the business logic generation process is repeated 10 times for each LLM and input configuration evaluated. LLMs use 0.5 as temperature for Claude, Qwen, and Llama models, or medium as reasoning effort for GPT models, which support the reasoning parameter instead of temperature.

Evaluation Method. We employ both a quantitative benchmark and a qualitative analysis of the generated controller code.

Quantitative benchmark. Previous studies have shown that generating test cases from Gherkin scenarios is feasible, but not yet

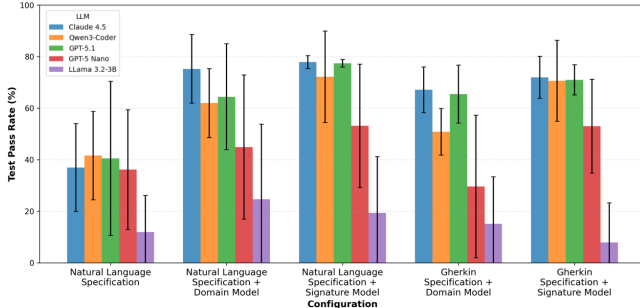


Figure 2: Overview of the Average Test Pass Rate

flawless [5, 31]. For higher reliability, we use a human-made, verified Python unit test suite for our study as shown by the green box in Figure 1. Each test function corresponds to a specific Gherkin scenario. During evaluation, we execute the backend system against this test suite. The primary metric for evaluating the generation quality quantitatively is the test pass rate (TPR) from executing the test suite against the generated business logic.

Qualitative Analysis. We conduct qualitative analysis through manual inspection of the generated business logic across different input configurations, as shown by the brown arrow in Figure 1. The primary goal is to identify recurring failure reasons for LLM-generated business logic and to evaluate code quality attributes missed in the quantitative benchmark.

3 Results and Analysis

Figure 2 shows the overall test pass rate (TPR) across all systems, modeling tools, and repetitions, grouped by configurations and Large Language Models (LLMs) used. From the graph, it is clear that adding any type of structured specifications significantly improves output quality compared to the baseline where only the natural language specification is provided.

Furthermore, while output generated using Gherkin specifications + any model has an overall lower accuracy than the ones generated using natural language specifications + any model (with 6.8% lower TPR and 0.9% higher standard deviation averaged across all combinations), the average TPR of Gherkin specifications outputs beats natural language outputs in 14 out of the 30 combinations we tested (5 LLMs $\times$ 2 modeling tools $\times$ 3 samples), with their average improvement being +7.7%. This result suggests that there is potential to further improve this approach to surpass natural language in output quality.

LLMs also show good out-of-the-box ability to infer system specifications through domain models, though providing the model layer’s signatures generated from the domain model instead improves average TPR (+7.82%) and lowers standard deviation (-2.47%).

Figure 2 also shows that most LLMs and configurations show high standard deviations in TPR. Manual inspection reveals that these fluctuations are largely attributable to the stochastic nature of LLMs, specifically regarding their ability to avoid common types of errors, which tend to propagate consistently throughout the entire generated controller when any of them manifests, resulting in batches of test failures. We also experiment with (in addition to the results presented) setting the temperature to 0 for LLMs permitting temperature adjustment, and still find similar trends.

Table 2 shows a closer view of TPR for individual samples, taking Claude as an example, since it is the overall best-performing model, with its highest configuration average TPR over 80% for 4 out of 6 samples and perfect TPR for several individual runs. From the results, the combination of domain model + Gherkin specifications shows highest average TPR in 3 out of the 6 samples we tested with Claude. Across all LLMs, signature model + natural language specifications shows highest average TPR in 11 out of the 30 LLM-modeling-tool-sample combinations, followed by domain model + Gherkin specifications with 9 out of the 30 combinations.

However, for both CheECSEManager samples, the domain model and Gherkin specification configuration result in anomalously low average TPR for Claude. Such performance degradations are observed in several configurations across other samples and LLMs, although the specific combinations triggering these outliers vary. Further investigation is required to isolate the underlying causes of these degradations in specific scenarios.

Table 3 shows the observed major error types (>1% of all errors) in the generated controllers that most commonly cause tests to fail. We note that more than 70% of failures (that is, everything listed except the third row) are caused by errors that can be detected post-generation through static code analysis tools.

Given these results, we argue that structured specifications have the potential to improve LLM repository-level code generation quality beyond the naïve and the natural language approaches, and that SSDE offers a concrete, viable path towards high-quality, verifiable, repository-level code generation.

4 Roadmap

Improving Output Quality. Our results indicate that while *structured spec-driven engineering* (SSDE) is promising, the approach can be improved in several aspects.

Incorporating *code analysis* and *feedback loops* is the most immediate path to improving output quality. Our analysis reveals that over 70% of the failures encountered during the pilot study are detectable through standard static analysis. Since these errors involve structural or type inconsistencies rather than logic flaws, an automated pipeline could parse the abstract syntax tree of the generated code against the model layer and feed these errors back to the LLM [1]. Other types of errors such as constraint validation can also be detected through test scenarios, which can be incorporated into feedback with dynamic analysis [11]. Given that existing work shows that Large Language Models (LLMs) possess self-correcting capabilities when provided with specific error context [7, 11, 19], this feedback loop could autonomously resolve the bulk of generation issues without human intervention.

We also observe that although Python is dynamically typed, the *use of type hints* helps provide more guidance to LLMs [8, 20]. We notice that data type mismatches account for >20% of test failures during our pilot study. We encourage software automation tools maintainers (e.g., umple and pycoregen) to include comprehensive type hints in their coding tools, to ensure these frameworks are LLM-ready and optimized for automated engineering workflows.

There is also potential for *LLM domain knowledge injection* via fine-tuning or advanced prompting to elucidate the mapping between domain model syntax and Python model layer syntax. Although domain models and their corresponding generated model

Table 2: Test Pass Rate of the Generated Business Logic with Claude Sonnet 4.5 using Various Input Configurations

System	Modeling Tool	Natural Language Specification			Gherkin Specification	
		(No Model)	Domain Model	Signature Model	Domain Model	Signature Model
Symboleo	Umple	0.0% $\pm$ 0.0%	79.9% $\pm$ 0.0%	79.9% $\pm$ 0.0%	99.1% $\pm$ 2.9%	79.9% $\pm$ 0.0%
	Ecore	47.9% $\pm$ 41.2%	79.9% $\pm$ 0.0%	79.9% $\pm$ 0.0%	81.7% $\pm$ 6.1%	79.9% $\pm$ 0.0%
CheECSEManager	Umple	43.5% $\pm$ 3.0%	73.0% $\pm$ 3.9%	76.7% $\pm$ 0.0%	25.7% $\pm$ 7.6%	79.2% $\pm$ 0.3%
	Ecore	16.8% $\pm$ 1.7%	53.2% $\pm$ 31.8%	67.1% $\pm$ 4.0%	26.0% $\pm$ 18.7%	28.2% $\pm$ 19.9%
MeetingGroups	Umple	81.6% $\pm$ 0.0%	82.2% $\pm$ 6.4%	85.0% $\pm$ 2.8%	83.4% $\pm$ 6.2%	84.6% $\pm$ 0.7%
	Ecore	31.9% $\pm$ 4.8%	82.4% $\pm$ 3.0%	78.8% $\pm$ 3.9%	84.2% $\pm$ 0.9%	79.8% $\pm$ 0.9%

Table 3: Distribution of Major Error Types for Failed Tests

Error Type	% of All Errors
Invoking non-existent API	49.0%
Data type mismatch	20.2%
Did not validate a given constraint	11.5%
API positional argument mismatch	3.2%
Referencing non-existent variables	1.0%

layer code theoretically convey equivalent information, using the latter yields higher test pass rate at the cost of increased input token consumption. This observation suggests that, while current LLMs exhibit proficiency with native Python code, they *lack specific domain knowledge* regarding model code generation tools such as umple and pyecoregen. By equipping LLMs with the ability to infer the expected model layer API directly from compact domain models, we anticipate a further improvement in LLM repository-level generation quality and a substantial reduction in performance fluctuation at lower cost.

Evaluating Real-World Impact. Assessing the test pass rate reveals only one aspect of SSDE’s contribution to the software engineering process. To rigorously evaluate SSDE’s practical utility, future work must quantify other benefits of SSDE automation.

In practice, LLM-enabled engineers often rely on *tooling support from coding agents* (e.g. GitHub Copilot [16], Claude Code [2]) through iterative conversations and feedback to improve code quality, rather than single-shot LLM calls for coding. To truly assess the viability of SSDE, experiments must extend beyond naïve input-output flow and evaluate the abilities of LLM agentic tools in repository-level code generation.

We are also actively working towards a *larger, more comprehensive SSDE dataset and benchmark* to evaluate LLM’s output quality using different types of inputs with reduced bias in terms of problem domains, variation, and complexity. However, existing software repositories with a complete set of natural language specifications, Gherkin specifications, and domain models are not readily available. While our pilot study uses three such systems obtained from GitHub, advancing the field requires the curation of a larger, more complex dataset to ensure generalizability and to mitigate engineering bias from constructing the inputs for specific experiments. Additionally, future evaluations must expand beyond simple correctness metrics, including the cost-effectiveness of token consumption, sustainability concerns and the output latency associated with different LLMs, behavioural models and input configurations.

Future research should also quantify the *net productivity gain of SSDE*. Barring groundbreaking innovations, LLM software repository generation will likely remain imperfect in the foreseeable future, necessitating manual intervention by engineers [4]. In addition, while structured specifications enable greater automation, their reliance on specific syntax and strict structure imposes an inherent *maintenance cost*. Though we envision some of these tasks to be less burdensome in the future if they can be automated via LLMs [5, 9, 31], future research should measure the effort required to produce structured specifications and fix generation errors based on test failures compared to the baseline of manually implementing the system and other automation approaches, such that their difference is the net productivity gain of SSDE in improving efficiency.

Applicability. SSDE offers a promising approach to extend LLM-automated software engineering to complex software systems and repositories. Still, specific limitations must be addressed to ensure SSDE is easily applicable for general software engineering use.

First, future research must investigate methods for *partial repository updates* in response to evolving specifications. Modern software is characterized by continuous evolution, yet due to the high variability of current LLM outputs, there is currently no guarantee that an update on the specification can be accurately reflected on the repository code. To transform SSDE to a solution for continuous maintenance, it is critical to develop partial code identification and generation techniques [14, 25], which enables the targeted propagation of specification updates to the existing codebase, ensuring systems can evolve while preserving the value of the existing code.

Second, given the diverse array of specification languages and software models tailored to specific software concerns, SSDE’s role in integrating with a variety of state-of-the-art and future technologies should be explored. As the automation of modeling and specification becomes more accessible, we should investigate how this approach can extend beyond standard software engineering and seek the potential to model complex representations of software systems or even physical objects, enabling LLMs and agentic tools to interpret and interact with the physical world.

Lastly, other than improving output quality through feedback, the verifiability of structured specifications can also be leveraged to build a variety of practical *software quality tools*. For instance, since Gherkin scenarios are inherently executable, it facilitates the automatic synthesis of *accept tests* and their *harnesses*, a capability we expect to mature as LLMs and software engineering technologies evolve. Furthermore, by treating structured specifications as the ground truth, we can build specialized *static analysis agents*

and custom *linters* that validate generated code against the specifications. This ecosystem of tools effectively shifts the engineer's role from code reviewer to specification architect, allowing for a scalable and reliable adoption of repository-level automation.

5 Conclusion

We present a vision for moving from file-level Large Language Model (LLM) coding assistance to repository-level automation through *structured spec-driven engineering* (SSDE). Rather than relying on purely natural language workflows, which suffer from inherent ambiguity and lack of verifiability, we argue that the path forward lies in leveraging decades of engineering knowledge that led to structured specifications like Gherkin and software models. Our pilot study generating Model-View-Controller systems shows that structured inputs have the potential to enable high quality repository generation. While challenges remain, we observe great potential to improve the output quality, such as 70% of errors from the naïve approach can be detected to enable automated corrections. We outline a research roadmap focused on quality, impact, and applicability to improve this approach. Ultimately, we argue that SSDE offers a promising vision for reducing manual engineering efforts and enabling high-level automation of engineering design.

6 Acknowledgement

This research was supported by Natural Science and Engineering Research Council of Canada (NSERC) through grant RGPIN-2025-04993.

References

- [1] Seyed Moein Abtahi and Akramul Azim. 2025. Augmenting Large Language Models with Static Code Analysis for Automated Code Quality Improvements. In *IEEE/ACM 2nd International Conference on AI Foundation Models and Software Engineering (FORGE'25)*. doi:10.1109/Forge66646.2025.00017
- [2] Anthropic. 2025. *Claude Code*. <https://code.claude.com/>
- [3] Anthropic. 2025. Introducing Claude Sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>.
- [4] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. (2025). arXiv:2507.09089 [cs.AI]
- [5] Severin Bergsmann, Alexander Schmidt, Stefan Fischer, and Rudolf Ramler. 2024. First Experiments on Automated Execution of Gherkin Test Specifications with Collaborating LLM Agents. In *Proceedings of the 15th ACM International Workshop on Automating Test Case Design, Selection and Evaluation (A-TEST'24)*. ACM, 12–15. doi:10.1145/3678719.3685692
- [6] Fatma Bozyigit, Tolgahan Bardakci, Alireza Khalilipour, Moharram Challenger, Guus Ramackers, Önder Babur, and Michel R. V. Chaudron. 2024. Generating domain models from natural language text using NLP: a benchmark dataset and experimental comparison of tools. *Software and Systems Modeling* 23, 6 (2024). doi:10.1007/s10270-024-01176-y
- [7] Boqi Chen, Aren A. Babikian, Shuzhao Feng, Dániel Varró, and Gunter Mussbacher. 2025. LLM-based Satisfiability Checking of String Requirements by Consistent Data and Checker Generation. In *33rd IEEE International Requirements Engineering Conference (RE'25)*. IEEE, 231–243. doi:10.1109/RE63999.2025.00030
- [8] Boqi Chen, José Antonio Hernández López, Gunter Mussbacher, and Dániel Varró. 2025. The Power of Types: Exploring the Impact of Type Checking on Neural Bug Detection in Dynamically Typed Languages. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE'25)*. IEEE Press, 489–501. doi:10.1109/ICSE55347.2025.00088
- [9] Kua Chen, Yujing Yang, Boqi Chen, José Antonio Hernández López, Gunter Mussbacher, and Dániel Varró. 2023. Automated Domain Modeling with Large Language Models: A Comparative Study. In *ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS'23)*. IEEE Press. doi:10.1109/MODELS58315.2023.00037
- [10] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. *Computing Research Repository* (2021). arXiv:2107.03374 [cs.LG]
- [11] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. arXiv:2304.05128 [cs.CL]
- [12] Cucumber. 2025. Gherkin. <https://github.com/cucumber/gherkin/>.
- [13] Den Delimarsky. 2025. Spec-driven development with AI: Get started with a new open source toolkit. GitHub. <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
- [14] Lukas Fruntke and Jens Krinke. 2025. Automatically Fixing Dependency Breaking Changes. *Proceedings of the ACM on Software Engineering*, Article FSE096 (2025). doi:10.1145/3729366
- [15] Antonio García-Domínguez and Dimitris Kolovos. 2024. EMFatic: A textual syntax for EMF Ecore models. <https://eclipse.dev/emfatic/>.
- [16] GitHub. 2021. *GitHub Copilot*. <https://github.com/features/copilot>
- [17] Sean Grove. 2025. The New Code. the AI Engineer World's Fair 2025. <https://www.youtube.com/watch?v=8rABwKRsec4>
- [18] Kamil Grzybek et al. 2019. Modular Monolith with DDD. <https://github.com/kgrybek/modular-monolith-with-ddd>.
- [19] Sirui Hong et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The 12th International Conference on Learning Representations (ICLR'24)*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [20] Faizan Khan, Boqi Chen, Daniel Varro, and Shane McIntosh. 2022. An Empirical Study of Type-Related Defects in Python Projects. *IEEE Transactions on Software Engineering* (2022). doi:10.1109/TSE.2021.3082068
- [21] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS'22)*. Curran Associates Inc., Article 1613, 22199–22213. <https://dl.acm.org/doi/10.5555/3600270.3601883>
- [22] Timothy C. Lethbridge et al. 2021. Umple: Model-driven development for open source and education. *Science of Computer Programming* 208 (2021). doi:10.1016/j.scico.2021.102665
- [23] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *The 37th Conference on Neural Information Processing Systems (NeurIPS'23)*. <https://openreview.net/forum?id=1qv610Cu7>
- [24] Meta AI. 2024. llama3.2:3b. <https://ollama.com/library/llama3.2:3b/>.
- [25] Behrooz Omidvar Tehrani, Ishaani M, and Anmol Anubhai. 2024. Evaluating Human-AI Partnership for LLM-based Code Migration. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA'24)*. ACM. doi:10.1145/3613905.3650896
- [26] OpenAI. 2025. GPT-5 nano Model. <https://platform.openai.com/docs/models/gpt-5-nano>.
- [27] OpenAI. 2025. GPT-5.1: A smarter, more conversational ChatGPT. <https://openai.com/index/gpt-5-1/>.
- [28] Santiago Padron, Julien Valentin, Iwan Olier, Rémi Séguin, Mathieu Guimond, Yejia Shen, and Daniel Yu. 2025. CheECSEManager. <https://github.com/F2025-ECSE223/ecse223-group-project-p16>. GitHub.
- [29] Mike Pagel, Vincent Aranega, and Andreas Schmidl. 2021. pycoregen. <https://github.com/pycore/pycoregen/>.
- [30] PlantUML. 2026. PlantUML at a Glance. <https://plantuml.com/>.
- [31] Alexander Poth, Olsi Rrjoll, Huiyu Wang, and Klaus Schmid. 2026. Baseline Evaluation of LLM-Facilitated UI Test-Case Generation from Gherkin Specifications. In *Systems, Software and Services Process Improvement*, Murat Yilmaz, Paul Clarke, Andreas Riel, Richard Messnarz, Mikus Zelmenis, and Ivi Anna Buce (Eds.). Springer Nature Switzerland. doi:10.1007/978-3-032-04288-0_3
- [32] Rohith Pudari and Neil A. Ernst. 2023. From Copilot to Pilot: Towards AI Supported Software Development. arXiv:2303.04142 [cs.SE]
- [33] Qwen Team. 2025. Qwen3-Coder. <https://github.com/QwenLM/Qwen3-Coder>.
- [34] Motez Saad, Boqi Chen, José Antonio Hernández López, Dániel Varró, and Tushar Sharma. 2025. Hierarchical Evaluation of Software Design Capabilities of Large Language Models of Code. (2025). arXiv:2511.20933 [cs.SE]
- [35] Sepehr Shariif, Alireza Parvizimosaei, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. 2020. Symboleo: Towards a Specification Language for Legal Contracts. In *IEEE 28th International Requirements Engineering Conference (RE'20)*. doi:10.1109/RE48521.2020.00049 Artifact URL: <https://github.com/Smart-Contract-Modelling-uOttawa/Symboleo-JS-Core>.
- [36] Jonathan Silva, Qin Ma, Jordi Cabot, Pierre Kelsen, and Henderik A. Proper. 2024. Application of the Tree-of-Thoughts Framework to LLM-Enabled Domain Modeling. In *Proceedings of the 43rd International Conference on Conceptual Modeling (ER'24)*. Springer-Verlag, 94–111. doi:10.1007/978-3-031-75872-0_6
- [37] David Steinberg, Frank Budinsky, Marcelo Paternostro, and Ed Merks. 2009. *EMF: Eclipse Modeling Framework 2.0* (2nd ed.). Addison-Wesley Professional. <https://dl.acm.org/doi/10.5555/1197540>
- [38] Artem Syromiatnikov and Danny Weyns. 2014. A journey through the land of model-view-design patterns. In *Proceedings of the 2014 IEEE/IFIP Conference on Software Architecture (ICSA'14)*. IEEE. doi:10.1109/WICSA.2014.13
- [39] Yujing Yang, Boqi Chen, Kua Chen, Gunter Mussbacher, and Dániel Varró. 2024. Multi-step Iterative Automated Domain Modeling with Large Language Models. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS'24)*. ACM, 587–595. doi:10.1145/3652620.3687807